

LMS Portal — Backend & SQL Complete Reference

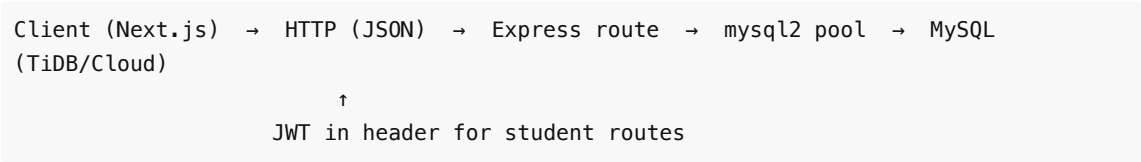
Project: 3D Websoft LMS Portal
Stack: Node.js · Express 5 · MySQL (mysql2 pool) · JWT · bcrypt
Source: lms-backend/server.js · lms-backend/db/index.js
Port: 9400 (or process.env.PORT on Render)

How to use this document

1. **Learn the pattern, not the line numbers** — Most endpoints repeat: validate body → optional JWT → run SQL with ? placeholders → return JSON.
2. **Memory trick — SQL verbs:**
 - o **SELECT** = read / check / list
 - o **INSERT** = create new row
 - o **UPDATE** = change existing row
 - o **DELETE** = remove row
 - o **JOIN** = combine tables using a shared key (usually id)
3. **Always use ? placeholders** — Never put user input inside the SQL string. That stops SQL injection.
4. **JWT flow** — Login returns token; protected routes read Authorization: Bearer <token> , verify with jwt.verify , use decoded.id as student_id .

Part 1 — Architecture overview

Request flow



Environment variables

Variable	Purpose
jwtsecret	Signs and verifies JWT tokens
host, port, user, password, database	MySQL connection (see db/index.js)
PORT	HTTP port in production

Database tables (logical model)

Table	Role
student	Learner accounts
adminlogin	Admin accounts (separate from students)
course1	Course catalog (shared content)

enrollment	Links student ↔ course + progress pct
course_module	Lessons per course (position, content)
module_progress	Which modules a student completed
attendance	Attendance rows (note spelling in DB)
result2	Grades / exam outcomes
exam1	Exam metadata per course
questions	MCQ text per exam
options	Choices per question (ischoice = correct flag)

Design idea: Course content is **global** (`course1` , `course_module`). **Enrollment** decides who sees it. Progress lives in `enrollment.pct` and `module_progress` .

Part 2 — SQL building blocks (remember forever)

2.1 `SELECT * FROM table WHERE column = ?`

What it does: Fetches rows matching one condition.

When to use:

- Login: find user by email
- Duplicate check before INSERT: “does this email already exist?”
- Fetch one profile by id

In this project:

- Register/login for `student` and `adminlogin`
- Enrollment duplicate check

Mental model: “Find rows where this column equals this value.”

```
SELECT * FROM student WHERE email = ?
```

General apps: Almost every “look up by unique field” (email, username, SKU).

2.2 `INSERT INTO table (cols...) VALUES (?, ?, ?)`

What it does: Adds a new row.

When to use: Registration, creating courses, marking attendance, adding questions.

Returns: `result.insertId` in Node — the new row’s auto-increment id.

Mental model: “Create one new record with these column values.”

```
INSERT INTO student (username, email, password) VALUES (?, ?, ?)
```

General apps: Any “Create” API (POST).

2.3 SELECT specific columns (not *)

What it does: Returns only safe/useful fields (e.g. hide password).

When to use: /me endpoint — return id, username, email only.

```
SELECT id, username, email FROM student WHERE id = ?
```

General apps: User profile APIs; GDPR-friendly responses.

2.4 INNER JOIN (or JOIN)

What it does: Combines two tables where keys match.

When to use: “Show courses for this student” — student is in enrollment , course details in course1 .

```
SELECT course1.*, enrollment.pct
FROM enrollment
JOIN course1 ON enrollment.course_id = course1.id
WHERE enrollment.student_id = ?
```

Remember:

- **FROM** = start table (often the “link” table: enrollment)
- **JOIN** = attach related table
- **ON** = how they connect (course_id = id)
- **WHERE** = filter to current user

General apps: Orders + products, posts + authors, enrollments + courses.

2.5 LEFT JOIN

What it does: Keeps all rows from the left table; matching right rows or NULL.

When to use: Admin lists where some foreign keys might be missing; questions with optional options.

```
SELECT questions.id, questions.question, options.id AS options_id, options.choice
FROM questions
LEFT JOIN options ON questions.id = options.question_id
WHERE questions.exam_id = ?
```

Mental model: “Show every question; attach options if they exist.”

General apps: Reports, admin dashboards, optional relations.

2.6 COUNT(column) AS alias

What it does: Counts rows (often with WHERE).

```
SELECT COUNT(status) AS total FROM aendance WHERE status = ?
```

When to use: Dashboard stats (present/absent counts).

Caveat in project: `/attendance-present` and `/attendance-absent` count **globally**, not filtered by `decoded.id` — worth fixing later for per-student stats.

General apps: Analytics, KPIs, “how many orders today?”

2.7 UPDATE ... SET ... WHERE ...

What it does: Changes existing rows.

```
UPDATE course_module
SET title = COALESCE(?, title), content = ?
WHERE id = ?
```

- `COALESCE(?, title)` — If new title is NULL, keep old title.
- Check `result.affectedRows === 0` → nothing matched → 404.

General apps: Edit profile, edit lesson content, change order status.

2.8 UPDATE with expressions — LEAST(pct + 5, 100)

What it does: Increments progress but caps at 100.

```
UPDATE enrollment SET pct = LEAST(pct + 5, 100)
WHERE course_id = ? AND student_id = ?
```

When to use: Gamification / completion percentage after each module.

General apps: Stock limits, rating caps, loyalty points max.

2.9 INSERT ... ON DUPLICATE KEY UPDATE

What it does: If a unique key conflict exists, update instead of error.

```
INSERT INTO module_progress (module_id, student_id, completed)
VALUES (?, ?, 1)
ON DUPLICATE KEY UPDATE completed = 1
```

Requires: Unique index on `(module_id, student_id)` (typical design).

When to use: “Mark done” clicked twice — idempotent save.

General apps: Upserts, favorites, cart lines, settings per user.

2.10 DELETE FROM ... WHERE ...

What it does: Removes rows.

When to use: Admin cleanup; must delete **children before parents** (options → questions → exam).

General apps: GDPR delete, admin panel, cascade deletes (or manual like this project).

2.11 SELECT id FROM ... WHERE id IN (?) (mysql2 expands arrays)

What it does: Finds which submitted option ids are correct.

```
SELECT id FROM options WHERE id IN (?) AND ischoice = ?
```

In submit flow: Compare count of correct selected options vs total answers → score % → grade → INSERT into `result2`.

General apps: Bulk validation, tag matching, cart item checks.

2.12 ALTER TABLE ... ADD COLUMN

What it does: Migrates schema at startup (simple approach).

```
ALTER TABLE course_module ADD COLUMN content TEXT NULL
```

When to use: Add `content` for lesson markdown without separate migration tool.

Production note: Prefer formal migrations (Flyway, Prisma migrate) for large teams.

2.13 ORDER BY

What it does: Sorts results.

```
ORDER BY course_module.position ASC
ORDER BY enrollment.id DESC
```

When to use: Lesson order; newest enrollments first.

Part 3 — Every endpoint and its queries

Health

Route	Method	SQL	Why
/hello	GET	None	Smoke test

Authentication & users

POST /register

1. SELECT * FROM student WHERE email = ?
 - **Why:** Prevent duplicate accounts.
 - **If rows:** 400 "already registered".
2. INSERT INTO student (username, email, password) VALUES (?, ?, ?)
 - **Why:** Create user; password is **bcrypt hashed** in Node, not in SQL.
 - **Then:** JWT with `insertId`.

Apply later: Same pattern for any signup with unique email.

POST /adminregister

Same as register but table `adminlogin` .

POST /login & POST /adminlogin

1. `SELECT * FROM ... WHERE email = ?`
2. `bcrypt.compare(password, row.password)` in Node
3. Issue JWT `{ id, name }`

Why not `WHERE email=? AND password=?` ? Passwords are hashed; SQL cannot compare plain text to hash.

GET /me

1. JWT verify
2. `SELECT id, username, email FROM student WHERE id = ?`

Why: Refresh UI profile without storing password in token.

GET /totalusers

- `SELECT * FROM student`
- **Why:** Admin dashboard count/list (returns full rows + `detailer = length`).

Courses & enrollment

POST /courses

1. `INSERT INTO course1 (name, description, duration, icon) VALUES (?, ?, ?, ?)`
 - **Why:** Course is shared catalog entry.
2. `INSERT INTO enrollment (student_id, course_id, pct) VALUES (?, ?, 0)`
 - **Why:** Assigns course to one student; `pct` starts at 0.

Design: Admin picks `student_id` when creating course in this API.

GET /totalcourses

- `SELECT * FROM course1` — list all courses for admin.

POST /enrollment

1. `SELECT * FROM enrollment WHERE student_id=? AND course_id=?`
 - **Why:** Avoid duplicate enrollment.
2. `INSERT INTO enrollment (student_id, course_id, pct) VALUES (?, ?, 0)`

GET /admin/enrollments

```
SELECT enrollment.id, enrollment.student_id, enrollment.course_id, enrollment.pct,
       student.username, student.email, course1.name AS course_name
FROM enrollment
JOIN student ON student.id = enrollment.student_id
JOIN course1 ON course1.id = enrollment.course_id
ORDER BY enrollment.id DESC
```

Why: One query for admin table: student name + course name + progress.

GET /courses (student, JWT)

```
SELECT course1.*, enrollment.pct
FROM enrollment
JOIN course1 ON enrollment.course_id = course1.id
WHERE enrollment.student_id = ?
```

Why: Student only sees **their** courses with progress — not entire catalog.

Attendance (aendance table)

POST /attendance

```
INSERT INTO aendance (student_id, course_id, Date, status) VALUES (?, ?, ?, ?)
```

Why: One row per mark (present/absent) per day per course.

GET /attendance (student, JWT)

```
SELECT aendance.student_id, aendance.course_id, aendance.Date, aendance.status,
       course1.id, course1.name
FROM aendance
JOIN course1 ON aendance.course_id = course1.id
JOIN student ON aendance.student_id = student.id
WHERE student.id = ?
```

Why: Show attendance with human-readable course name.

GET /attendance-present / attendance-absent

```
SELECT COUNT(status) AS total FROM aendance WHERE status = ?
```

Why: Quick counts for UI (see caveat: not scoped to student in present route).

Results

POST /result

```
INSERT INTO result2 (course_id, student_id, assesment, score, grade, status) VALUES
(?, ?, ?, ?, ?, ?)
```

Why: Manual grade entry by admin.

GET /result (student, JWT)

```
SELECT result2.course_id, result2.student_id, result2.assesment, result2.score,
       result2.grade, result2.status, course1.name
FROM result2
```

```
JOIN course1 ON course1.id = result2.course_id
JOIN student ON student.id = result2.student_id
WHERE student.id = ?
```

Why: Student transcript with course names.

Exams, questions, options, submit

POST /exam

```
INSERT INTO exam1 (course_id, questionnumbers, duration) VALUES (?, ?, ?)
```

Why: Exam shell linked to a course.

GET /exam (student, JWT)

```
SELECT exam1.id, exam1.course_id, exam1.questionnumbers, exam1.duration, course1.name
FROM exam1
JOIN course1 ON course1.id = exam1.course_id
JOIN enrollment ON enrollment.course_id = exam1.course_id
WHERE enrollment.student_id = ?
```

Why: Only exams for courses the student is enrolled in.

POST /questions

```
INSERT INTO questions (exam_id, question) VALUES (?, ?)
```

GET /questions (query exam_id, JWT)

```
SELECT questions.id, questions.question, questions.exam_id,
       options.id AS options_id, options.choice, options.ischoice
FROM questions
LEFT JOIN options ON questions.id = options.question_id
WHERE questions.exam_id = ?
ORDER BY questions.id
```

Why: One HTTP call returns flat rows; frontend groups by question id.

POST /options

```
INSERT INTO options (question_id, choice, ischoice) VALUES (?, ?, ?)
```

Why: `ischoice` stored as 0/1 for correct answer.

POST /submit (JWT)

1. `SELECT id FROM options WHERE id IN (?) AND ischoice = 1` — score correct picks.
2. `SELECT course_id FROM exam1 WHERE id = ?` — link result to course.

3. `INSERT INTO result2 (...) VALUES (...)` — store Exam #<id> , score, grade, pass/fail.

Grade logic (in Node): A+ ≥95, A ≥90, B ≥80, ... F below 50; status Fail if score < 50.

Modules & progress

POST /modules

```
INSERT INTO course_module (course_id, title, position, content) VALUES (?, ?, ?, ?)
```

Fallback without `content` column if DB is old.

PUT /modules/:id

```
UPDATE course_module SET title = COALESCE(?, title), content = ? WHERE id = ?
```

GET /modules (JWT, query course_id)

```
SELECT course_module.id, course_module.course_id, course_module.title,
       course_module.position, course_module.content
FROM course_module
JOIN enrollment ON course_module.course_id = enrollment.course_id
WHERE enrollment.course_id = ? AND enrollment.student_id = ?
ORDER BY course_module.position ASC
```

Why: Security — student only gets modules for courses they're enrolled in.

POST /modules/completed (JWT)

Chain:

1. `SELECT course_id FROM course_module WHERE id = ?`
2. `SELECT completed FROM module_progress WHERE module_id=? AND student_id=?` — skip if already done.
3. `INSERT ... ON DUPLICATE KEY UPDATE completed=1`
4. `UPDATE enrollment SET pct = LEAST(pct+5, 100) WHERE ...`
5. `SELECT pct FROM enrollment WHERE ...` — return new % to UI.

Why: Tie lesson completion → progress bar.

GET /modules/completed

```
SELECT module_progress.module_id, module_progress.completed, course_module.title
FROM module_progress
JOIN course_module ON course_module.id = module_progress.module_id
JOIN enrollment ON enrollment.course_id = course_module.course_id
                AND enrollment.student_id = module_progress.student_id
WHERE module_progress.student_id = ?
      AND course_module.course_id = ?
      AND module_progress.completed = 1
```

Admin read-all routes

Route	Pattern
/admin/attendance	aendance LEFT JOIN course1, student
/admin/results	result2 LEFT JOIN course + student
/admin/exams	exam1 LEFT JOIN course1
/admin/modules	course_module LEFT JOIN course1 ORDER BY course, position
/admin/questions	questions LEFT JOIN exam1
/admin/options	options LEFT JOIN questions

Why LEFT JOIN: Show records even if a related name is missing (safer admin lists).

Admin delete routes (cascade order matters)

Delete student (DELETE /admin/students/:id):

1. module_progress → enrollment → aendance → result2 → student

Why: Foreign keys / orphan prevention.

Delete course (DELETE /admin/courses/:id):

1. For each exam: delete options → questions → exam1
2. For each module: delete module_progress → course_module
3. enrollment , aendance , result2 , course1

Other deletes: Single-table by id or composite key (attendance by student+course+Date; result by course+student+assessment+score).

Part 4 — Node.js patterns used

mysql2 connection pool

```
const dbConnect = sql.createPool({ host, port, user, password, database, ssl })
```

Why pool: Reuses connections under load (Render/serverless-friendly).

Callback style

```
dbConnect.query(sql, [params], (error, result) => { ... })
```

Async alternative in project: runQuery() Promise wrapper for admin deletes.

Security checklist

Practice	In project?
----------	-------------

Parameterized queries ?	Yes
Password hashing (bcrypt)	Yes
JWT for student routes	Mostly yes
Admin routes protected	No middleware — rely on frontend/admin only (improve for production)
CORS open	Yes — configure for your domain in production

Part 5 — Cheat sheet: “I need to build X”

I need...	SQL pattern
Unique signup	SELECT by email → INSERT
Login	SELECT by email → bcrypt in JS
User-specific list	JOIN link table + WHERE user_id = ?
Admin rich table	JOIN dimension tables + aliases (AS course_name)
Prevent duplicates	SELECT count/rows → then INSERT
Idempotent save	INSERT ON DUPLICATE KEY UPDATE
Increment with cap	UPDATE SET col = LEAST(col + n, max)
MCQ grading	SELECT ids WHERE IN (?) AND flag = correct
Safe delete tree	DELETE children deepest-first
Lesson order	ORDER BY position ASC

Part 6 — Quick revision (5-minute drill)

1. Name the **link table** between students and courses. → `enrollment`
2. Why JOIN `enrollment` in `GET /courses` ? → Filter to logged-in student.
3. Why bcrypt? → Never store plain passwords.
4. What does `insertId` give after INSERT? → New row primary key.
5. What does `ON DUPLICATE KEY UPDATE` prevent? → Error on second “mark done”.
6. Why delete `options` before `questions` ? → Child rows first.
7. `LEFT JOIN` vs `JOIN` ? → LEFT keeps all left rows; INNER only matches.
8. What is `?` for? → Parameter binding / SQL injection safety.

Part 7 — Complete SQL catalog (every statement in `server.js`)

Below, each query is numbered. **Reuse rule:** If two endpoints use the same SQL, learn it once.

Q1 — Duplicate email check (student)

```
SELECT * FROM student WHERE email = ?
```

Used in	POST /register
Parameters	[email] from body
Success signal	result.length === 0 → safe to insert
Failure signal	result.length > 0 → 400 already registered
Real-world	Signup forms, newsletter subscribe, username availability
Remember	"SELECT before INSERT" = cheap uniqueness without DB unique index (you should still add UNIQUE(email) in production)

Q2 — Create student

```
INSERT INTO student (username, email, password) VALUES (?, ?, ?)
```

Used in	POST /register
Parameters	[username, email, hashedPassword]
Node detail	bcrypt.hash(password, 10) — 10 = cost factor (2^10 rounds)
After insert	result.insertId → JWT id and response studentId
Real-world	Any user creation row

Q3 — Duplicate email check (admin)

```
SELECT * FROM adminlogin WHERE email = ?
```

Same pattern as Q1, table `adminlogin`, route `POST /adminregister`.

Q4 — Create admin

```
INSERT INTO adminlogin (username, email, password) VALUES (?, ?, ?)
```

Same pattern as Q2 for admins.

Q5 — Login lookup (admin)

```
SELECT * FROM adminlogin WHERE email = ?
```

Used in	POST /adminlogin
Note	Code passes [email, password] to query but SQL only uses ? once — password check is only in bcrypt.compare
Empty result	401 invalid credentials
Real-world	Standard login: one SELECT by unique identifier

Q6 — Login lookup (student)

```
SELECT * FROM student WHERE email = ?
```

Same as Q5 for POST /login .

Q7 — List all students

```
SELECT * FROM student
```

Used in	GET /totalusers
Why *	Admin wants full list (includes password hash — avoid exposing this to public clients)
Count trick	result.length in Node instead of SELECT COUNT(*)
When COUNT is better	Millions of rows — never pull all rows just to count

Q8 — Current user profile (safe columns)

```
SELECT id, username, email FROM student WHERE id = ?
```

Used in	GET /me
ID source	decoded.id from JWT
Why not *	Never send password hash to browser
Real-world	/api/users/me in any app

Q9 — Create course (catalog)

```
INSERT INTO course1 (name, description, duration, icon) VALUES (?, ?, ?, ?)
```

Used in	POST /courses
Design	Course exists once; many students enroll via enrollment
insertId	Becomes course_id for next query

Q10 — Enroll student in new course

```
INSERT INTO enrollment (student_id, course_id, pct) VALUES (?, ?, 0)
```

Used in	POST /courses (step 2), POST /enrollment
pct = 0	Progress bar starts empty
Real-world	Junction / bridge table row linking two entities

Q11 — List all courses

```
SELECT * FROM course1
```

Used in: GET /totalcourses (admin catalog).

Q12 — Check duplicate enrollment

```
SELECT * FROM enrollment WHERE student_id = ? AND course_id = ?
```

Used in	POST /enrollment
Composite condition	Both IDs must match same row
Real-world	"Already in cart", "Already following"

Q13 — Admin enrollment report (multi-JOIN)

```
SELECT enrollment.id, enrollment.student_id, enrollment.course_id, enrollment.pct,  
       student.username, student.email, course1.name AS course_name  
FROM enrollment  
JOIN student ON student.id = enrollment.student_id  
JOIN course1 ON course1.id = enrollment.course_id  
ORDER BY enrollment.id DESC
```

Used in	GET /admin/enrollments
JOIN chain	enrollment → student (names), enrollment → course1 (title)
AS course_name	Avoid column name clash if both tables had name
ORDER DESC	Newest enrollments on top
Mnemonic	Start at fact table (enrollment), attach dimensions (student, course)

Q14 — Student's courses + progress

```
SELECT course1.*, enrollment.pct
FROM enrollment
JOIN course1 ON enrollment.course_id = course1.id
WHERE enrollment.student_id = ?
```

Used in	GET /courses (JWT)
Filter	decoded.id
Star on course1	All course fields + extra pct from enrollment
Real-world	"My orders", "My playlists"

Q15 — Insert attendance

```
INSERT INTO aendance (student_id, course_id, Date, status) VALUES (?, ?, ?, ?)
```

Used in	POST /attendance
Date	Column name Date — reserved word; quoted in MySQL if needed
status	Typically present / absent strings
Real-world	Event logs, check-ins

Q16 — Student attendance history

```
SELECT aendance.student_id, aendance.course_id, aendance.Date, aendance.status,
       course1.id, course1.name
FROM aendance
JOIN course1 ON aendance.course_id = course1.id
JOIN student ON aendance.student_id = student.id
WHERE student.id = ?
```

--	--

Used in	GET /attendance
Extra JOIN student	Redundant filter could be WHERE aendance.student_id = ? only; current form still works
Purpose	Attach course1.name for UI

Q17 — Count attendance by status

```
SELECT COUNT(status) AS total FROM aendance WHERE status = ?
```

Used in	/attendance-present, /attendance-absent
Parameter	'present' or 'absent'
Improvement	Add AND student_id = ? for per-student stats
COUNT(*) vs COUNT(status)	For non-null status both behave similarly

Q18 — Insert result (manual)

```
INSERT INTO result2 (course_id, student_id, assesment, score, grade, status) VALUES (?, ?, ?, ?, ?, ?)
```

Used in: POST /result , POST /submit (auto after exam).

Q19 — Student results with course name

```
SELECT result2.course_id, result2.student_id, result2.assesment, result2.score,
       result2.grade, result2.status, course1.name
FROM result2
JOIN course1 ON course1.id = result2.course_id
JOIN student ON student.id = result2.student_id
WHERE student.id = ?
```

Used in: GET /result .

Q20 — Create exam

```
INSERT INTO exam1 (course_id, questionnumbers, duration) VALUES (?, ?, ?)
```

Used in: POST /exam . Links exam to parent course.

Q21 — Exams visible to enrolled student

```
SELECT exam1.id, exam1.course_id, exam1.questionnumbers, exam1.duration, course1.name
FROM exam1
JOIN course1 ON course1.id = exam1.course_id
JOIN enrollment ON enrollment.course_id = exam1.course_id
WHERE enrollment.student_id = ?
```

Key idea	Enrollment is the authorization filter — no exam for courses you're not in
Real-world	Content gated by subscription/enrollment

Q22 — Add question

```
INSERT INTO questions (exam_id, question) VALUES (?, ?)
```

Used in: `POST /questions` .

Q23 — Questions + options (flat result)

```
SELECT questions.id, questions.question, questions.exam_id,
       options.id AS options_id, options.choice, options.ischoice
FROM questions
LEFT JOIN options ON questions.id = options.question_id
WHERE questions.exam_id = ?
ORDER BY questions.id
```

LEFT JOIN	Question with zero options still appears
Frontend	Group rows by questions.id to build nested JSON
ORDER BY	Stable question order

Q24 — Add MCQ option

```
INSERT INTO options (question_id, choice, ischoice) VALUES (?, ?, ?)
```

ischoice: `1` = correct, `0` = wrong (Node: `ischoice ? 1 : 0`).

Q25 — Grade exam answers

```
SELECT id FROM options WHERE id IN (?) AND ischoice = ?
```

Used in	<code>POST /submit</code>

IN (?)	mysql2 expands array of selected option ids
Second ?	1 = only correct options
Scoring	correct = result.length, total = answers.length

Q26 — Resolve course from exam

```
SELECT course_id FROM exam1 WHERE id = ?
```

Used in: POST /submit before inserting into result2 .

Q27 — Insert module

```
INSERT INTO course_module (course_id, title, position, content) VALUES (?, ?, ?, ?)
```

Fallback (no content column):

```
INSERT INTO course_module (course_id, title, position) VALUES (?, ?, ?)
```

Q28 — Update module

```
UPDATE course_module SET title = COALESCE(?, title), content = ? WHERE id = ?
```

COALESCE: New title optional; content can be set to empty string.

Q29 — List modules (enrollment guard)

```
SELECT course_module.id, course_module.course_id, course_module.title,
       course_module.position, course_module.content
FROM course_module
JOIN enrollment ON course_module.course_id = enrollment.course_id
WHERE enrollment.course_id = ? AND enrollment.student_id = ?
ORDER BY course_module.position ASC
```

Security: Both course_id (query param) and student_id (JWT) must match enrollment.

Q30 — Module → course lookup

```
SELECT course_id FROM course_module WHERE id = ?
```

Used in: POST /modules/completed to know which enrollment.pct to update.

Q31 — Already completed?

```
SELECT completed FROM module_progress WHERE module_id = ? AND student_id = ?
```

If `completed === 1` : Return early — no double progress bump.

Q32 — Upsert module progress

```
INSERT INTO module_progress (module_id, student_id, completed) VALUES (?, ?, 1)
ON DUPLICATE KEY UPDATE completed = 1
```

Requires unique key on `(module_id, student_id)` .

Q33 — Bump enrollment progress

```
UPDATE enrollment SET pct = LEAST(pct + 5, 100) WHERE course_id = ? AND student_id = ?
```

+5 per module — business rule in SQL cap with `LEAST` .

Q34 — Read progress after update

```
SELECT pct FROM enrollment WHERE course_id = ? AND student_id = ?
```

Returns fresh `pct` to frontend.

Q35 — List completed modules in course

```
SELECT module_progress.module_id, module_progress.completed, course_module.title
FROM module_progress
JOIN course_module ON course_module.id = module_progress.module_id
JOIN enrollment ON enrollment.course_id = course_module.course_id
                AND enrollment.student_id = module_progress.student_id
WHERE module_progress.student_id = ?
      AND course_module.course_id = ?
      AND module_progress.completed = 1
```

Double enrollment join: Ensures student still enrolled when listing completions.

Q36 — Schema migration at boot

```
ALTER TABLE course_module ADD COLUMN content TEXT NULL
```

Ignores “duplicate column” error if already applied.

Q37–Q42 — Admin SELECT lists

ID	SQL essence	Route
Q37	aendance LEFT JOIN course1, student	/admin/attendance
Q38	result2 LEFT JOIN course1, student	/admin/results
Q39	exam1 LEFT JOIN course1	/admin/exams
Q40	course_module LEFT JOIN course1 ORDER BY course, position	/admin/modules
Q41	questions LEFT JOIN exam1	/admin/questions
Q42	options LEFT JOIN questions	/admin/options

Q43–Q58 — Admin DELETE statements

Query	Purpose
DELETE FROM module_progress WHERE student_id=?	Clear progress before student delete
DELETE FROM enrollment WHERE student_id=?	Unlink courses
DELETE FROM aendance WHERE student_id=?	Clear attendance
DELETE FROM result2 WHERE student_id=?	Clear grades
DELETE FROM student WHERE id=?	Remove student
SELECT id FROM exam1 WHERE course_id=?	Find exams to cascade
SELECT id FROM questions WHERE exam_id=?	Find questions per exam
DELETE FROM options WHERE question_id=?	Remove choices
DELETE FROM questions WHERE exam_id=?	Remove questions
DELETE FROM exam1 WHERE course_id=?	Remove exams
SELECT id FROM course_module WHERE course_id=?	Find modules
DELETE FROM module_progress WHERE module_id=?	Progress per module
DELETE FROM course_module WHERE course_id=?	Lessons
DELETE FROM enrollment/aendance/result2 WHERE course_id=?	Course-scoped data
DELETE FROM course1 WHERE id=?	Remove course
DELETE FROM enrollment WHERE id=?	Single enrollment row
DELETE FROM aendance WHERE student_id=? AND course_id=? AND Date=?	Precise attendance row
DELETE FROM result2 WHERE id=? or composite WHERE	Remove grade rows

DELETE FROM exam1/questions/options/modules by id	Granular admin deletes
---	------------------------

Cascade mantra: *Leaves first (options), then branches (questions), then trunk (exam/course).*

Part 8 — Full REST API index

Method	Path	Auth	Main tables
GET	/hello	No	—
POST	/register	No	student
POST	/adminregister	No	adminlogin
POST	/adminlogin	No	adminlogin
POST	/login	No	student
GET	/totalusers	No	student
GET	/me	JWT student	student
POST	/courses	No*	course1, enrollment
GET	/totalcourses	No	course1
POST	/enrollment	No*	enrollment
GET	/admin/enrollments	No*	enrollment + joins
GET	/courses	JWT	enrollment, course1
POST	/attendance	No*	aendance
GET	/attendance	JWT	aendance, course1
GET	/attendance-present	JWT ⁺	aendance
GET	/attendance-absent	JWT ⁺	aendance
POST	/result	No*	result2
GET	/result	JWT	result2, course1
POST	/exam	No*	exam1
GET	/exam	JWT	exam1, enrollment
POST	/questions	No*	questions
GET	/questions	JWT	questions, options
POST	/options	No*	options
POST	/submit	JWT	options, exam1, result2
POST	/modules	No*	course_module

PUT	/modules/:id	No*	course_module
GET	/modules	JWT	course_module, enrollment
POST	/modules/completed	JWT	module_progress, enrollment
GET	/modules/completed	JWT	module_progress
GET	/admin/*	No*	various
DELETE	/admin/*	No*	various

*Production should protect admin/write routes.

†Token verified but query not filtered by student in present/absent counts.

Part 9 — Study flashcards (read aloud once daily)

1. **Enrollment** = who can see which course.
2. **JOIN** = glue tables on matching ids.
3. **WHERE after JOIN** = filter rows (user, date, status).
4. **INSERT + insertId** = create then link child rows.
5. **SELECT before INSERT** = duplicate guard.
6. **bcrypt** = hash at register; compare at login.
7. **JWT** = stateless session; `decoded.id` = current student.
8. **ON DUPLICATE KEY** = safe repeat clicks.
9. **LEAST(col+n, max)** = increment with ceiling.
10. **DELETE children first** = no orphan FK errors.

Generated for LMS Portal backend reference. Update this file when `server.js` changes.